

USENIX

THE ADVANCED COMPUTING
SYSTEMS ASSOCIATION

Software Availability Protection in Cyber-Physical Systems

Ao Li, Jinwen Wang, and Ning Zhang, *Washington University in St. Louis*

https://www.usenix.org/conference/usenixsecurity25/presentation/li-ao

This paper is included in the Proceedings of the
34th USENIX Security Symposium.

August 13–15, 2025 • Seattle, WA, USA

978-1-939133-52-6

Open access to the Proceedings of the
34th USENIX Security Symposium is sponsored by USENIX.

ARTIFACT EVALUATED
usenix ASSOCIATION
AVAILABLE

Software Availability Protection in Cyber-Physical Systems

Ao Li*, Jinwen Wang*, Ning Zhang
Computer Security and Privacy Laboratory
Washington University in St. Louis

Abstract

Existing efforts in software protection have mostly focused on how to detect violations of confidentiality or integrity, with the goal of safeguarding information or ensuring the correctness of execution. Little has been done to study the handling of such violations, where the common practice is to crash the program. However, such strategies sacrifice availability, which is not acceptable in real-time safety-critical cyber-physical systems (CPSs), where untimely computation can have catastrophic physical-world consequences.

To bridge this gap, we present Gecko, an attack recovery approach that not only timely recovers the execution from the attack but also disables exploited features to improve system availability. Realizing Gecko presents two technical challenges. To defend against repeated exploitation, Gecko utilizes compartmentalization for runtime attack input identification and introduces fail-safe shadow compartments to disable the exploited features while ensuring graceful degradation. To remove attack impacts in a timely manner, Gecko employs selective data reset through snapshot recovery. It further uses an I/O reference monitor to avoid peripheral re-configuration. We developed a prototype of Gecko and evaluated it on three CPS platforms: ArduPilot, Jackal UGV, and OpenManipulator. Gecko achieves recovery with 83.3% task deadline hits while incurring a runtime overhead of 8.28%.

1 Introduction

As computing technology becomes ubiquitous across different societal sectors, CPSs have become a cornerstone of modern infrastructure. While this integration has transformed our daily lives, it also underscores the importance of safeguarding these systems. However, existing software security protections, such as memory encryption [1] and control-flow integrity (CFI) [2] focus on the protection of confidentiality and integrity. Upon detection of security violations, the common approach is to terminate and restart the process. However, this design choice has direct impacts on system availability, which is crucial for CPSs that continuously interact with the physical world, as correct but untimely computational results can have catastrophic physical-world consequences.

Existing Availability Solutions and Open Problems. While existing software security defenses [2–4] often regard availability as a less critical property, there have been significant efforts in system resiliency [5–10], which is closely related. Unfortunately, these approaches often assume a specific fault model for achieving system availability. As such, they face fundamental challenges when applied to security attacks. First, some approaches aim to maintain execution continuity after a crash [11, 12] by addressing the root cause. However, they are designed for specific bugs (e.g., concurrency bugs) under benign scenarios, where the patterns of bug manifestations and root causes are known. However, the adaptive nature of attack often renders these methods less effective. Second, existing approaches often focus on the timely recovery of cyber data, with little attention to peripherals (connection to the physical world). They typically reset corrupted data via either rebooting [5,7,8,11] or checkpoint/restore [13,14]. The former often leads to long periods of unresponsiveness, while the latter is less applicable to CPSs that rely on external peripheral states [15], as these states cannot be restored simply by resetting processor memory.

Our Solution – Gecko. To bridge the gap, we design and develop Gecko[1], a recovery-based approach to improve real-time software availability in safety-critical real-time CPS against memory corruption attacks. In this work, we formulate real-time software availability as the ability of tasks/processes to meet deadlines in real-time CPSs. Under the assumption that there exists a reference monitor that can detect violations of security properties, Gecko focuses on real-time availability in the handler. Therefore, the availability protection we seek necessitates tackling two hard problems, maintaining system availability (*liveness*) against repeated exploitation attempts

*equal contribution.

[1]The name is inspired by the gecko's self-amputation behavior, where it sheds its tail to evade threats.

and recovering execution in a timely manner (*timeliness*).

Ensuring System Liveness - Mitigating Repeated Exploitation by Feature Reduction. To ensure system availability, simply restarting the system is insufficient, as the attacker can repeatedly exploit the vulnerability to crash the target. Ensuring system continuity (system liveness) requires isolating adversarial inputs that exploit vulnerabilities and compensating for the resulting loss of input, both of which are challenging problems in system and CPS security. To identify the adversarial input that triggered the vulnerability, we build on top of existing literature on causality analysis in crash [16, 17] and attack [18–24] triage. However, many of these methods are designed as analysis tools that trace large amounts of memory instructions across the entire program space and can incur prohibitive overhead for real-time systems. Gecko addresses this challenge by leveraging software compartmentalization to enable tracking of runtime dependencies with less overhead, albeit at the cost of coarser granularity. On the other hand, removing the exploited input will inadvertently change the underlying program, potentially impacting the control of the CPS. To address this, Gecko draws inspiration from fault recovery paradigms and introduces shadow compartments, which operate as a safe mode for the disabled features, facilitating a graceful degradation of performance.

Ensuring System Timeliness - Rapid Recovery via Selective State Restoration. Besides mitigating repeated exploitation, it is also important to recover the cyber-physical control process in a timely manner, especially for physical systems that require high-frequency control loops, such as fighter jets. Gecko builds on top of existing checkpointing solutions by storing a clean program state after system initialization and rolling back to this state upon recovery. While the checkpoint/restore process incurs minimal computational overhead, re-calibrating peripherals (e.g., sensors and actuators) after a data reset can result in significant delays. Building on the observation that sensors and actuators are often configured only during initialization and do not require reconfiguration afterward, Gecko leverages a reference monitor to prevent any unauthorized reconfigurations of these peripherals after the initial setup. Concretely, Gecko isolates the software modules that contain the interfaces for configuring the sensors and actuators, and restricts access to them as read-only after initialization. This approach prevents tampering with the configurations of the sensors and actuators, thereby avoiding the need for re-calibration after a reset.

Prototype and Evaluation. We implemented a prototype of Gecko on both ARM Cortex-A and x86 architectures. To evaluate the performance of Gecko, we deployed it on three real-world applications, namely a UAV (ArduPilot [25]) and an indoor UGV (Jackal [26]) in a simulated environment, as well as a real-world robotic arm (OpenManipulator X) [27]. The results show Gecko incurs an acceptable average runtime overhead of 8.28%. When the system is under attack, Gecko is able to identify and isolate the exploited input, maintaining the safety of the target system with a task deadline hit rate exceeding 80%.

Contributions. The paper contributes the following:

- We formulated the notion of real-time software availability, and show recovery from attacks needs to meet both liveness and timeliness requirements.
- We design and develop Gecko, a recovery technique to achieve real-time software availability, where repeated exploitations are thwarted by replacing the exploited features in the system with a fail-safe one, effectively disabling the attack vector. Gecko further improves the timeliness of the recovery by considering the peripherals that interconnect cyber and physical components.
- We implemented a prototype of Gecko[2] and evaluated it on three real-world CPSs, assessing its feasibility in both normal execution and attack scenarios, and analyzed the factors impacting its performance.

2 Motivation and Related Work

As CPSs interact with the physical world, their correctness depends on both functional and temporal correctness. CPS software is typically composed of *tasks*, which are assigned with precise timing constraints. Tasks are generally implemented as recurring or periodic workloads and serve as the minimal schedulable units, with each instance required to complete execution by a specified *deadline*. Failure to meet deadlines can result in catastrophic outcomes; for instance, a delayed control signal in self-driving cars could lead to accidents. Based on the severity of missed deadlines, tasks can be categorized as either *hard*, *firm*, or *soft*. Hard implies that a violation is considered a system failure. Firm indicates that violating the timing constraint is highly undesirable. Soft means that occasionally violating the timing constraint is acceptable, but negatively impacts the quality of the system. In this paper, we primarily focus on soft real-time systems, as they represent the majority of real-world industrial cases [28].

In the remainder of this section, we formulate the problems and discuss the limitations of existing works. To further motivate this paper, we present a preliminary evaluation highlighting their insufficiencies. The evaluation was conducted on four representative CPS platforms: the PX4 Autopilot [29] and ArduPilot [25] drones, the OpenManipulator X [27] robotic arm, and the Jackal [26] UGV. Jackal is tested on an Intel NUC 8. ArduPilot and OpenManipulator X were tested on a Raspberry Pi 3b (Cortex-A). PX4 was tested on a Pixhawk (Cortex-M). To ensure timing fidelity, we conducted experiments on hardware-in-the-loop (Jackal) or on native

[2]Source code is publicly available at https://github.com/WUSTL-CSPL/Gecko.

hardware with real sensors (ArduPilot, PX4, OpenManipulator). The more detailed experimental setup is provided in Section 6.

2.1 Problem Formulation

Due to the continuous interaction with the physical world, availability is a necessity for safety. In this work, we formulate real-time software availability as the ability of tasks/processes to meet deadlines in real-time cyber-physical systems. In the context of recovery-based availability protection, real-time software availability can be decomposed into *liveness* and *timeliness*.

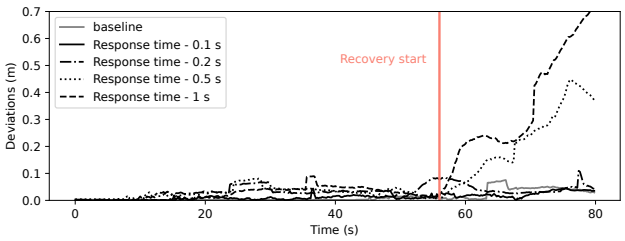
Liveness. Many software faults are typically transient or one-time issues, which a reboot can often mitigate. In contrast, in the case of adversarial attacks, a determined attacker can repeatedly exploit the same vulnerability after a reboot as long as the vulnerability remains present. Such difference in adversarial behavior makes the liveness problem fundamentally different from conventional recovery-based fault tolerance. In this paper, we consider a system to be live if it is capable of eventually completing the execution.

Timeliness. To ensure safety, many CPSs must respond quickly to changes in the environment. A delayed response can cause physical damage and/or loss of life. As such, recovery should be able to react within the tolerable threshold. In this paper, we consider timeliness to be the system's ability to recover from a single attack within a known period that can still allow the system to maintain safety. This represents a significant distinction from traditional systems, such as data-center environments, where delayed responses primarily affect service objectives rather than having life-critical consequences. To highlight the importance of timeliness, we conducted a preliminary evaluation of the impact of response time on control performance using the Jackal. To quantify the impact on control, we use the deviation between the actual and reference paths as the metric. Figure 1a shows the control deviations under different recovery response times of Jackal UGV. It can be observed that the vehicle can only maintain within a bound of 0.2 seconds recovery time; otherwise, it deviates from its reference path by more than 0.4 meters, potentially leading to a collision in the physical world.

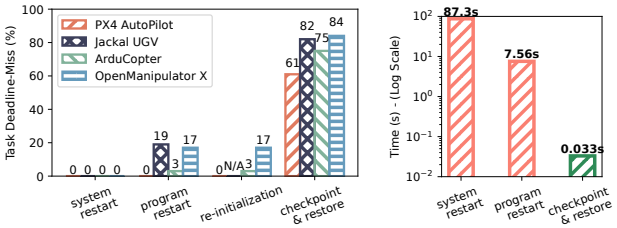
2.2 Existing Solutions

Given the goal of Gecko, related approaches can be categorized into two groups: one focuses on preventing repeated attacks (liveness) and the other aims to eliminate the attack impacts promptly (timeliness). The summary information[3] is presented in Table 1.

[3]While the goal is attack recovery, we also include existing techniques for fault recovery for a comprehensive solution space.



(a) Control deviations at different recovery response times.



(b) Deadline hits. (c) Response times.

Figure 1: Preliminary study results.

Ensuring Execution Continuity. Many prior works have explored techniques that maintain software execution in the presence of faults or attacks. *Execution Environment Manipulation:* There are many non-transient faults that program restart cannot resolve, as these faults persist until the underlying faulty component is repaired. Rx [11] identifies the external environmental conditions that lead to such faults and manipulates these conditions, such as adjusting thread interleavings, to avoid a crash. However, this approach is effective only for a limited scope of faults, where the bug manifestation and root causes are known. The adaptive nature of attacks renders this method ineffective. *N-Version Programming:* This line of research introduces additional implementation versions as redundancy to respond to faults or attacks. Existing approaches can be further divided into software-based [32,36] and hardware-based redundancy [33,37], both of which are costly as they require running multiple instances simultaneously. Hardware-based methods further demand hardware modifications. Moreover, employing redundancy as a recovery mechanism requires identifying problematic components, which is feasible in fault scenarios but challenging in the face of attacks. *Crash-resilient Instruction Set:* ZeRØ [31] introduces unique memory instructions that trigger a handler upon a security violation. However, it also requires hardware modifications and necessitates developers to redesign the controller to respond to security violations. *Malicious Input Filtering.* Some works are dedicated to filtering malicious inputs upon detecting an attack [30, 38, 39]. However, these techniques rely on dynamic tracing during runtime, making them prohibitively expensive. For instance, Bouncer [38] uses dynamic symbolic execution, Vigilante [30] employs dynamic dataflow analysis, and VSEF utilizes dynamic taint analysis [39].

Table 1: Current Approaches for Attack and Fault Recovery

	Example reference	Target scenarios	Attack/Fault model		**Timeliness**	**Liveness**	Engineering efforts	
			Model	Vector			w.o. Hw modification	w.o. controller redesign
Ensuring Execution Continuity								
Execution environment manipulation	Rx [11]	general program	fault	-	●		●	
Malicious Input filtering	Vigilante [30]	general program	attack	-		◓	●	
Crash-resilient instruction set	ZeRØ [31]	general program	attack	memory	●			
N-Versions	A^8 [32]	general program	attack	memory	●		●	
	BlueBox [33]	CPS	attack	-	●			
Re-randomization	Fired [34]		attack	memory	●		●	
Recovering from Faults								
Program restart	Huang et al. [35]	general program	fault	-		N/A	●	●
Microreboot	Otherworld [5, 7]	kernel drivers	fault	-		N/A	●	
Re-execution	ConAir [12]	general program	fault	-		N/A	●	●
Checkpoint/restore	Koo et al. [13]	general program	fault	-	◓	N/A	●	●
	Kong et al. [14]	CPS	attack	sensor	◓	N/A	●	
Gecko	This work	CPS	attack	memory	●	●	●	◓

Re-randomization: Fired [34] employs re-randomization to thwart repeated exploitation; however, while randomization can prevent targeted payload exploitation, it does not necessarily prevent software crashes caused by exploits. *Control-based Restart:* Abdi et al. [40] proposed using frequent restarts to mitigate repeated attacks. It leverages control laws to synthesize controllers that ensure safety under repeated attacks. However, this approach is limited to stateless controllers, whereas real-world controllers are typically stateful.

Recovering from Faults. There is also a large body of work that aims to recover from faulty states. *System/Program restart:* A strawman solution is to reboot the system [40] or the program [11], a technique widely used in software rejuvenation [35]. However, in our investigation, rebooting the system and program on the Jackal UGV takes 87.3 seconds and 3.56 seconds, respectively (as shown in Figure 1c), both of which violate the timeliness requirements. As seen in Figure 1b, most tasks on the three platforms fail to meet their deadlines. *Microreboot:* A more advanced approach is microreboot [5, 8, 41–43], which focuses on the feasibility of restarting only specific system components to reduce restart time. However, in real-time systems, the most basic schedulable unit is a task, which is not divisible because dividing it would require redesigning and re-verifying the scheduling. Consequently, the minimum rebootable unit is a task, which takes 3.56 seconds to restart in the Jackal UGV. (as shown in Figure 1c). *Re-execution:* This approach [12] recovers the program by re-executing from a previous program point. However, this is effective for idempotent code regions, while most CPS are stateful due to their recurrent workload nature. *Checkpoint/restore:* From our experiments, checkpointing the program states and then rolling back to clean states incurs an acceptable overhead. However, checkpoint/restore can only reset cyber-states, while CPSs are composed of external peripheral devices (i.e., sensors and actuators). Once these devices are corrupted, it can take several seconds to reinitialize them. From Figure 1b, it can be observed that this re-initialization causes most tasks (over 83%) to miss their deadlines.

Summary. Prior work on ensuring execution continuity cannot guarantee liveness properties under attacks due to several factors, including the unpredictable nature of adversarial behavior, prohibitive runtime costs, and substantial engineering efforts. The research question of how to effectively remove the attack surface at runtime remains largely unstudied. Moreover, existing program state recovery techniques do not adequately achieve timeliness due to the need for re-calibration of CPS peripherals.

3 Threat Model and Security Goal

We assume there are one or more remotely exploitable vulnerabilities in the CPS program, which is typically a user-space program. While we do not make any assumptions on the location and type of the vulnerability, we do assume that there exists a security reference monitor that can detect exploitation of the vulnerability, which manifests as a violation of the security policy the reference monitor enforces. Note the assumption on the exploitation detection is essential for recovery since vulnerability identification is equivalent to the halting problem and is undecidable.

The security guarantee Gecko seeks to establish is real-time software availability (intuitively, the defender regains full control of the system under a reduced feature set within a bounded time) when the attacker is actively attempting the exploitation of the vulnerability (causing denial of service). While we do believe our solution has the potential to generalize to diverse bug classes, our evaluation and analysis in this work focus specifically on memory corruption vulnerabilities, where the attacker has arbitrary memory reads and writes in the program space. We consider the following attacks out of the scope of this work: malicious hardware corruptions, sensor attacks [44], side-channel attacks [45], confused deputy

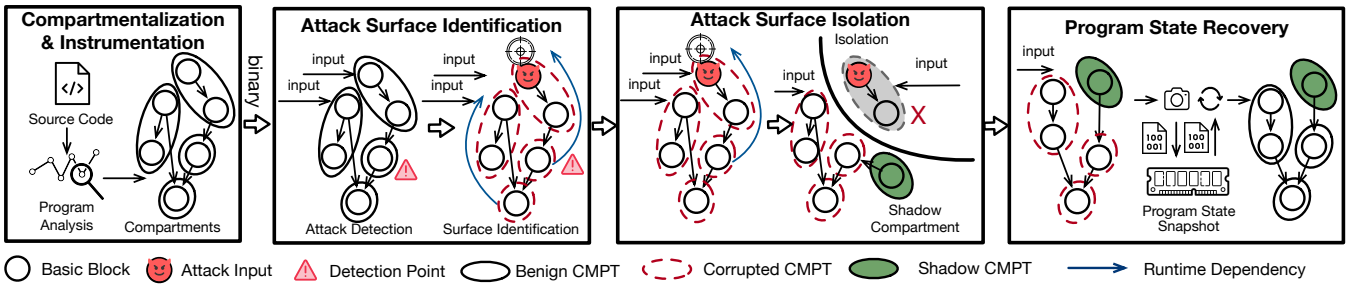


Figure 2: System workflow of Gecko.

attacks [46, 47], and performance interference attacks [48].

The design and implementation of Gecko further makes the following assumptions. In line with other works [3, 49, 50] on memory security, we assume that code injection is prevented using modern memory protection technology (e.g., MMU or MPU), therefore instrumentation cannot be tampered by the attacker. Since our primary target is user-space security, the privileged software components (e.g., OS and hypervisor, etc), hardware (e.g., computation devices, storage, and I/O peripherals, etc) and Gecko's own components are trustworthy. We also assume that developers have access to the source code, which is required for analysis and instrumentation. We assume the underlying attack detector is capable of identifying specific types of information flow violation [51, 52], which depends on the properties the system is designed to protect. More specifically, the reference monitor in our prototype can only detect control-flow violations [53] and value-based data-flow violations [54].

4 Design of Gecko

Key Idea – Leveraging Feature Reduction upon Recovery to Ensure System Availability. Many CPSs operate in safety-critical scenarios, thus fault tolerance is a common design necessity to ensure safety. This is also a compliance requirement in industrial standards, such as the EN 50128 standard for railway software [55] and ECSS-Q-HB-60-02A from the European Space Agency [56].

Faults can be categorized into transient or permanent [57]. Unlike transient faults, which can be mitigated by rebooting, permanent faults persist until the faulty component is repaired or replaced. In such cases, recovery from a permanent fault requires redundancy, which serves as an alternative to replace the original faulty component. Systematically, redundancy can be identical replicas or feature-degraded versions such as fail-safe mechanisms. For example, in ArduPilot, the former can be the case that the software maintains tracks for multiple inertial measurement units (IMUs) [58], each serving as a backup for the others. For the latter, the ArduPilot codebase implements 11 well-defined fail-safe modes [59]. Figure 3 shows a simplified code example, where the drone triggers

```
01 void Copter::failsafe_gcs_on_event(void) {
02     Failsafe_Action desired_action;
03     switch (g.failsafe_gcs) { // check failsafe configuration
04         case FS_GCS_ENABLED_ALWAYS_LAND:
05             desired_action = Failsafe_Action_Land;
06             break;
07         default: // Default failsafe action
08             desired_action = Failsafe_Action_RTL;
09       /* . . . */
10     // Call the failsafe action handler
11     do_failsafe_action(desired_action, ModeReason::GCS_FAILSAFE); }
```

Figure 3: A simplified fail-safe code snippet in ArduPilot.

fail-safe action when the connection to the ground station fails. More generally, Table 2 summarizes the availability of fault tolerance in the four CPS platforms examined in Section 2. In fact, all four platforms incorporate fail-safe mechanisms, such as fail-safe landing in PX4 [60] and replanning behavior in Jackal [61, recovery behavior] and OpenManipulator [62], to name a few.

Inspired by this, Gecko adopts redundancy in *attack* recovery. Upon attack, Gecko disables some functionalities along with the input vector and executes a secure reduced feature redundant version to ensure system availability. Specifically, it follows the principle of *using simplicity to control complexity* in the simplex architecture [36], where the system transitions to a simpler yet safer controller when faults are detected in the complex controllers.

Table 2: Fault Tolerance Availability in CPS Platforms

	ArduPilot	PX4	Jackal UGV	OpenManipulator
Replica	✓	✓	-	-
Fail-safe	✓	✓	✓	✓

High-Level Workflow. As shown in Figure 2, the workflow of Gecko has two phases: compile-time and runtime. At compile time, (1) the compiler performs static analysis on the program's source code and adds the instrumentation to enable compartmentalization and recovery. At runtime, (2) Gecko enforces the protection under normal conditions. If an attack is detected, it identifies the compartment associated with the attack. (3) Then, it reduces the attack surface by disabling the compartment receiving the malicious input and switching to a feature-reduced recovery block. This prevents attackers from

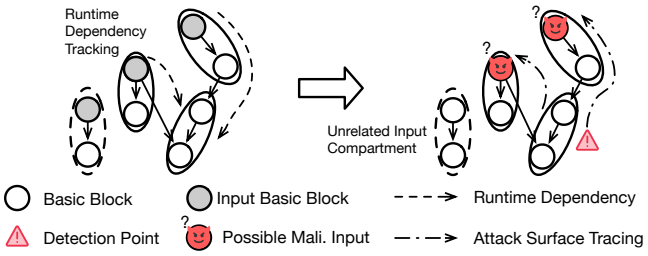


Figure 4: Compartment-level attack surface identification.

exploiting the same vulnerability while ensuring execution continuity. (4) The selective recovery mechanism rolls the corrupted program state back to a clean one and resumes the execution.

4.1 Program Compartmentalization

To prevent repeated attacks, Gecko chose to reduce the input attack surface by replacing the complex block with the feature-reduced recovery block. While intuitive, identifying the input attack surface of memory exploit is challenging. Existing memory exploit prevention, such as CFI and DFI, aim to detect policy violations which typically involve a sequence of events as part of the bug exploitation. However, these violations do not necessarily happen at the exact locations of the vulnerable program statements. Essentially, the attribution to the attack surface hinges on tracing the information flows from the inputs throughout the program execution, enabling the trace-back to the attack input source upon a security policy violation. Dynamic tainting [17] is effective for this purpose; however, its prohibitive overhead (37× slowdown) significantly limits its applicability in real-time and embedded environments, as it requires tracking every instruction dependency during runtime. To solve this challenge, Gecko leverages program compartmentalization to enable efficient information flow tracking, albeit with a trade-off in granularity. The high-level design is shown in Figure 4.

Compartmentalization and Isolation. Gecko partitions the program into individual compartments. A compartment is defined as an isolated code region, along with its accessible data, peripherals, and allowed control-flow transfers [63]. Gecko adopts a code-centric approach [64], where each instruction belongs to exactly one compartment, while data and peripherals may be accessible from multiple compartments.

The compartmentalization scheme is determined by the chosen policy, which varies based on the specific needs of the application. By default, Gecko adopts a task-based partitioning approach to address these requirements. This choice is based on two observations. First, tasks are generally the basic functional units in CPS. Second, the data within tasks are well self-contained. Consequently, a task-based compartmentalization policy introduces fewer compartment switches, reducing runtime delays.

The partitioning is conducted on program dependence graph (PDG) [65]. Specifically, it iteratively begins partitioning each compartment from the entry function of each task. It performs forward analysis and groups all the related code and data. When this forward analysis reaches code or data belonging to other compartments, it indicates that these elements are not exclusive but are shared among different compartments. Gecko moves them into a new separate compartment and grants access to the related compartments.

Concretely, data shared by multiple compartments are grouped into global compartments, and compartment stacks are allocated to separate regions to prevent cross-stack corruption via the stack pointer. Legitimate access to global data and cross-stack data is enabled by the reference monitor. At runtime, all indirect jumps and memory writes within compartments are sandboxed to prevent access to code and data outside the compartment. Inter-compartment control-flow and data-flow are trampolined to the compartment switch reference monitor, which resides outside the compartments and verifies their validity according to the PDG.

Attack Detector. The attack detection mechanisms are orthogonal to Gecko's design. In this paper, we adopt CFI and value-based DFI [54] within compartments. The value-based DFI instrument code before each write instruction, which records the written value (unlike traditional DFI, which records the instruction's identity). When the same memory is read, the instrumented code checks if the current value matches the previously recorded value, allowing it to detect any illegitimate write operations. Its advantage lies in enabling the selective verification of critical variables, thereby maintaining a manageable overhead. Notably, the nature of compartmentalization allows for the deployment of different detection methods across compartments. In practice, for less critical compartments, such as a drone's video recording functionality, users can opt not to enforce attack detection, further reducing overhead.

To prevent attacks that hijack control flow into infinite loops (thereby blocking other tasks from running on the CPU), Gecko 's detector monitors the execution time of each task. If a task exceeds its predefined worst-case execution time (WCET), the detector triggers an alarm.

4.2 Attack Surface Attribution

Gecko tracks runtime dependencies to attribute the attack input surface upon detecting an attack. To achieve a balance between runtime efficiency and attribution accuracy, it utilizes a multi-staged compartmentalization strategy as part of its attribution mechanism.

Multi-staged Attack Input Compartment Attribution. Concretely, the attack detector monitors for violations of the expected security policy at runtime. When an attack is de-

tected, Gecko traces back to the compartment(s) that received the malicious input based on runtime dependencies. The identified compartment(s) will then be isolated in the next phase (detailed in § 4.2), such that the entry point of the attack is removed. When the system is under execution, information flow, thus dependency, among the compartments is recorded by the compartment switch reference monitor. Gecko leverages this runtime context to attribute the detected attack from the compartment where it was identified to the compartment that received the malicious input.

However, attack surface attribution being performed at the compartment level is unavoidably over-approximated, often resulting in the attribution of the attack to a large number of unnecessary inputs. On the other hand, even though fine-grained compartmentalization allows for more precise attribution, the runtime overhead is prohibitive. To address this dilemma, Gecko employs multi-stage compartmentalization strategies where the granularity of compartments is increased upon attack detection for attribution. Under normal conditions, Gecko operates with the default coarser-grained compartmentalization. However, when a compartment containing multiple inputs is attributed as the attack surface, Gecko switches that compartment to the finer-grained version. Furthermore, there are compartments with large amounts of static dependency, Gecko further leverages the dynamic tracked compartment information flow between consecutive attack instances to narrow down the set of possible attack vectors.

Incremental Input-oriented Re-Compartmentalization. To enable seamless online switching, Gecko implements incremental re-compartmentalization, where the finer-grained version is built as an extension of the coarser-grained version. This approach enables selective switching to a finer-grained version for specific compartments, rather than completely overwriting the compartmentalization strategy for the entire program.

A natural question is how to determine the desired granularity for finer attribution. To this end, Mutable Protection Domains (MPDs) [66] model the cost of compartmentalization and formulate the compartmentalization policy as an optimization problem. This enables us to find a policy that maximizes isolation utility within deadline constraints. While MPDs can achieve optimal protection, they require accurately modeling timing costs of individual compartments and their communication costs, which is a challenging problem on modern computing platforms [67]. Gecko instead avoids this by special-purpose heuristics. Drawing on the insight that input separation is the most important factor driving the fidelity of attack attribution, Gecko further adopts an input-oriented compartmentalization strategy, partitioning compartments on a per-input basis. Building on top of the default task-level compartmentalization (described in the previous section), Gecko further partitions the compartment by inputs. Specifically, Gecko initially assigns each input statement to an individual sub-compartment. It then analyzes program dependencies

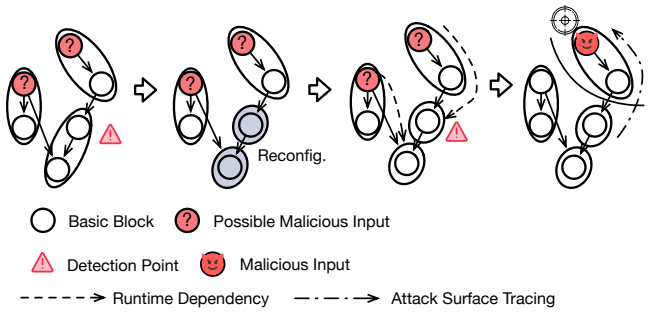


Figure 5: Multi-stage attack surface attribution.

from each input statement. It groups the sequential statements into a compartment until the next statement involves new input data flow dependencies.

4.3 Graceful Degradation via Shadow Compartment

To compensate for the disabled input(s) and reduced functionality, Gecko makes use of *Shadow Compartment*, which serves as a simplified replacement of the original compartment. When an original compartment is disabled, the shadow compartment swaps in and handles invocations from other compartments on behalf of the original compartment. The disabled inputs can be categorized as either non-essential or essential for maintaining control performance. Non-essential inputs, such as user interface inputs, primarily affect operability, while essential inputs, such as sensor data, are critical for control performance. Depending on the reduced inputs, Gecko provides two modes within the shadow compartment. The default mode ensures execution continuity for non-essential input reductions, while the customized mode handles reductions in essential inputs, aiming to achieve graceful degradation.

Default Mode. This mode is automatically supported and target-agnostic, eliminating the need for manual intervention. Concretely, the default mode monitors the return values of inter-compartment calls by recording the initial return value for each compartment pair. When activated, it directly provides the recorded value to the caller compartment. Returning default values in shadow compartments, although straightforward, is effective for maintaining execution continuity when certain inputs are non-essential for system control. For example, debugging and visualization features in user-machine interfaces can be removed without significantly affecting control performance, even though this removal reduces user interactivity with the target CPS. Nevertheless, prior research has identified these interfaces as high-likelihood attack vectors [68]. While sensor data can also be manipulated, it is less likely to lead to memory corruption due to its fixed data patterns (e.g., length).

Customized Mode. The customized mode handles the cases

Table 3: Existing Methods Compensating Sensor Unavailability

	Inputs		Model	Output	
	Sensor	Actuation		Sensor	Actuation
PID-Piper [73]		✓	ML model		✓
Software Sensor [71]		✓	State-space	✓	
Kalman Filter [72]	✓		State-space	✓	
UnRocker [70]	✓		ML model	✓	

where essential input data are missing. Following the principles of the Simplex architecture [36], this mode employs a simpler recovery block to replace the vulnerable component. Developing such a recovery block typically requires domain and application-specific knowledge. Talos [69] addresses this challenge by utilizing built-in error-handling mechanisms within applications as recovery blocks. As outlined in Section 2, mature CPS software often incorporates extensive built-in fail-safe mechanisms. For instance, ArduPilot includes 11 fail-safe modes. These existing codebases can be directly retrofitted by developers to function as shadow compartments. However, in the absence of such built-in error-handling code or when a more advanced safe mode is desired, additional manual effort is often necessary. This highlights the need for toolchains to support this process.

Support for Development. Although the development of shadow compartments is application-specific and requires domain expertise, Gecko aims to streamline this process by offering supporting tools. These tools are designed based on the development requirements identified in existing related research. Some of the existing work on input data compensation is summarized in Table 3. When partial input data becomes unavailable, these approaches leverage the remaining sensors and system states to either estimate the missing sensor measurements [70–72] or directly generate corrective control commands [73, 74]. This is typically accomplished using either state-space functions [71] or machine learning models [70, 73, 74], both of which are developed based on empirical data.

Based on this observation, Gecko facilitates the data collection during the shadow compartment development phase. The process of developing a shadow compartment in Gecko follows this pipeline: *(i)* Gecko identifies the compartments affected by a missing sensor input. It first identifies the compartments that include sensing tasks by recognizing the I/O accessing interfaces in the control loop. By analyzing data flow in the PDG, developers can determine the I/O task-dependent compartments and identify the type of data the policy should predict, i.e., the policy's output. *(ii)* Users execute various missions under a trusted environment for data collection. Gecko facilitates this procedure by providing built-in logging interfaces that record sensor and actuation data based on identified I/O tasks. *(iii)* With the collected data, users can build the predictive model for target data compensation offline and deploy it inside the shadow compartment, which is used in response to the input isolation.

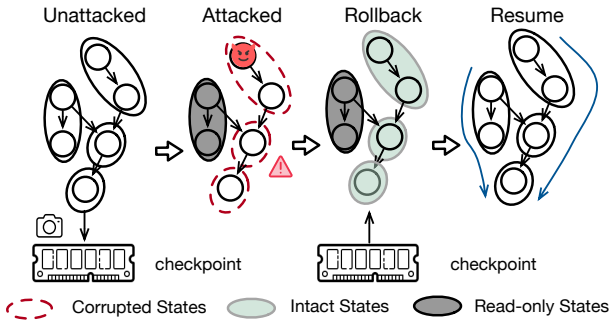


Figure 6: Checkpoint and restore in Gecko.

4.4 Attack Impacts Mitigation

Besides reducing the attack surface, it is necessary to mitigate the attack impacts. Gecko uses checkpoint/restore techniques for efficiency. An alternative method is lock-stepping [75], which runs multiple instances of a program synchronously and switches to a clean one upon recovery. However, this approach requires multiple resources (e.g., CPU) for running different program instances, and thus can be challenging to deploy in real-time embedded environments.

Program State Reset. As discussed in Section 2, it is challenging, if not impossible, to determine when a malicious payload enters the system without significant state recording. When using snapshots and rollback, attacks likely occur before detection. As such, unlike existing works [76, 77] that periodically checkpoints the memory states to handle the occasional faults, Gecko checkpoints once at the point when no malicious input has been received, i.e., immediately after system initialization.

Peripheral Re-Configuration Prevention. The attacker can also tamper with the memory write operations used to configure sensors and actuators. While rebooting can reinitialize the peripherals, it takes a considerable amount of time for reconfiguration, as demonstrated empirically in Section 2. Gecko leverages built-in compartmentalization to prevent malicious sensor and actuator configurations at runtime. As sensors and actuators are calibrated only during system initialization and remain unchanged, their driver tasks do not need dynamic inputs afterward. Consequently, Gecko partitions the functional calls for configuring sensors and actuators as separate compartments at compile-time. At runtime, Gecko ensures that sensor compartments will not allow any reconfigurations unless specifically permitted through the whitelist according to the security policy. Furthermore, the actuation compartment will only allow values from a permitted range that is known to be recoverable through subsequent updates. Furthermore, manually identifying the sensor and actuator configuration code requires intensive manual efforts. To address this issue, Gecko automatically identifies the sensor and actuator configuration functions and addresses through dynamic analysis.

5 Implementation

The implementation primarily consists of the following components.

Program Compartmentalization. This is implemented on top of the open-source project ACES [63] and mainly includes two steps: compartmentalization generation and enforcement.

Generation. Gecko employs an LLVM front-end analysis pass to identify compartment transitions, global data accesses, and cross-stack memory interactions between compartments. This analysis is developed based on SVF [78]. A task-based compartmentalization policy is adopted in our implementation, which is described in Section 4.1. Each source file containing a task from the task list forms a compartment. At linking time, a Python script is used to separate the code and data of the same compartment into different ELF sections and place them in a contiguous memory address space. The code sections are enforced as executable-only.

Enforcement. The trampoline code is instrumented as a reference monitor, replacing the identified statements. Sandboxing is achieved by masking the high bits of reserved registers storing the addresses of indirect jump and write instruction targets within a compartment. The mask is only modified before a global write instruction and in the reference monitor to access the global variable compartment and switch compartments. Runtime dependency between compartments is constructed by recording all global data accesses and inter-compartment calls. To support portability on both x86 and ARM architectures, the back-end passes are achieved by two distinct LLVM passes. These two passes mainly differ in analyzing architecture-specific indirect jump and write instructions. Additionally, LLVM automatically handles variable-length instruction encoding, ensuring seamless integration across architectures. As a user-space solution, Gecko is designed to operate independently of the memory isolation implementation from underlying OS and hardware, enabling deployment on computing platforms equipped with either an MMU or an MPU.

Attack Surface Detection and Isolation. The current prototype adopts built-in CFI [79] within the LLVM compiler and a value-based DFI, derived from OAT [54], to detect DFI violations. The value-based DFI protects control-dependent data, such as conditional variables, and semantically critical data, including input, output, and control variables. To measure the execution delay of each compartment, we instrument timestamp recording functions at the start and end of interface functions (i.e., functions invoked by or returning to other compartments) and compute the timestamp differences as the execution delay.

Data Reset Mechanism. This component primarily involves checkpoint/restore and prevention of the dynamic reconfiguration of peripherals.

State Checkpoint/Restore. This component is developed based on the codebase of the open-source project CRIU [80]. The snapshot copies data from each compartment's memory space and registers, storing it outside the sandbox. To address the memory boundaries, Gecko uses the PROVIDE() function in linker scripts to export the beginning and ending addresses of EFL sections for each compartment during linking.

Sensor and Actuator Configuration Prevention. Gecko disables sensor and actuator configurations by grouping configuration functions into separate compartments and preventing other compartments from invoking them after initialization. Since our evaluation platforms are Linux-based, Gecko locates these peripheral configuration functions by first narrowing down to functions that contain related syscalls, such as ioctl. For non-Linux platforms (e.g., low-cost microcontrollers), the focus shifts to memory-mapped I/O (MMIO) operations instead of ioctl invocations. Once confirmed via dynamic runs, these invocations of ioctl are replaced with reference monitors that check parameters.

6 Evaluation

We evaluate Gecko to answer the following questions:

- What are the costs of Gecko under no-attack conditions? (Section 6.1)
- How effective is Gecko at recovering from an attack? (Section 6.2)
- Does Gecko work on a real CPS platform? (Section 6.3)

Experimental Setup. The evaluation was conducted on three real-time CPS platforms:

ArduPilot Drone [25]. It is an open-source project widely used in UAVs. It includes over 30 tasks, encompassing typical UAV functionalities like navigation and control. We run it on a Navio2 [81], which uses a Raspberry Pi 3B as the computing platform along with a companion board that includes several onboard sensors. We run the ArduPilot software stack in software-in-the-loop simulation mode.

Jackal UGV [26]. It is an open-source platform commonly used for autonomous vehicle studies. We set up a hardware-in-the-loop simulation environment for experiments. The simulator runs on a host PC and the software stack of Jackal UGV runs on an Intel NUC 8. In the simulation, we attach a SICK LMS1xx LiDAR [82] to the Jackal UGV as the sensor for localization.

OpenManipulator X [27]. It is an open-source robotic arm equiped with the industrial-grade software stack, MoveIt [83]. MoveIt is widely adopted in both industry [84–86] and academia. We deploy it on a real robot arm, which uses a Raspberry Pi 3B for computation, a Pi Camera Module V2 for sensing images, and inertial measurement units at each joint to capture motion data.

For each platform, we manually extract the deadline of each task from the source code. For tasks where the deadline

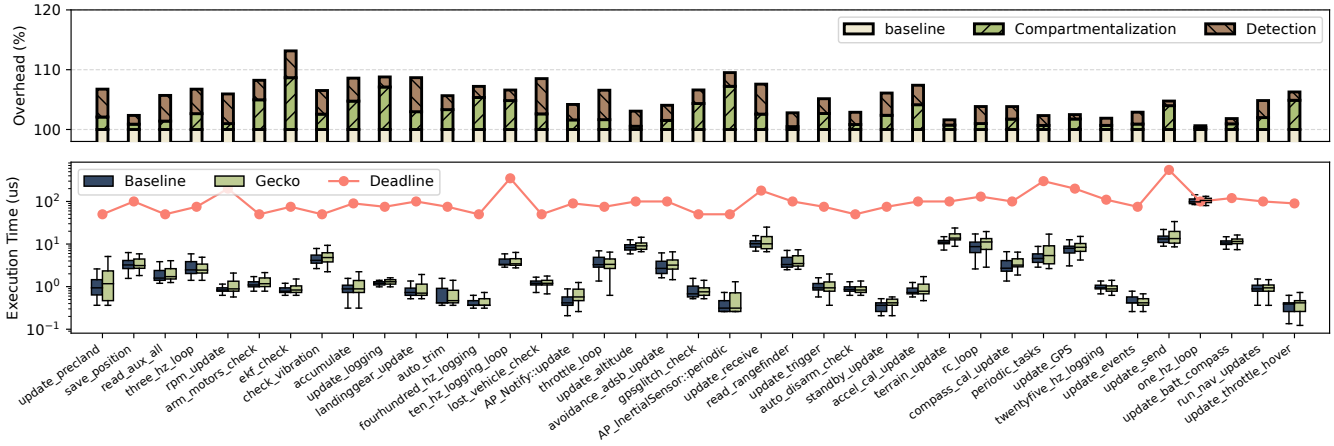


Figure 7: Runtime overhead breakdown under normal conditions on ArduPilot.

is not specified, we assume the deadline is the same as the period. This is common in real-time task models, also known as implicit deadlines or the Liu & Layland model [87].

6.1 Cost of Gecko

We evaluate the cost of Gecko by measuring the runtime and memory overhead of Gecko on ArduPilot and Jackal UGV. To evaluate the runtime overhead of software components in Gecko, we measured the execution times of real-time tasks in ArduPilot and Jackal over 100 runs for each software component. To measure the memory overhead of Gecko, we monitor applications' maximum Resident Set Size (RSS). Under task-based compartmentalization, ArduPilot is partitioned into a total of 39 task compartments, while the Jackal has 6 task compartments.

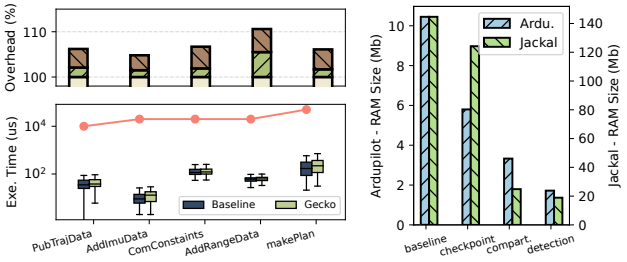
Runtime Overhead. The breakdown of the normalized runtime overhead for each task in ArduPilot and Jackal is shown in the top part of Figure 7. The average end-to-end runtime overhead is 8.28% in ArduPilot and 7.52% in Jackal. The bottom part of the figure shows the execution times and their variations, with each box representing the 25th percentile to the 75th percentile. Tasks exhibit varying execution time changes depending on their original workloads. In ArduPilot, the average standard deviations across all tasks are 3.43 μs without Gecko and 3.59 μs with Gecko. This indicates that Gecko slightly increases the variation, possibly due to its impact on data locality, such as increasing cache misses. However, the overall increase is only 4.7%. In the case of Jackal, the standard deviations are 68.9 μs without Gecko and 69.1 μs with Gecko, also indicating a marginal increase.

Besides, none of the tasks on either platform violate their deadline. The task closest to a deadline violation is one_hz_loop, likely due to its I/O-intensive logging workload. The native version is already close to the deadline, and Gecko only increases execution time by 3.12%.

Compartmentalization Overhead. This part of runtime overhead arises from two main causes: enforcing control-flow integrity and data-flow integrity between compartments. The former incurs overhead during compartment switches, while the latter occurs during memory access operations on shared data between compartments. As a result, the compartmentalization runtime overhead increases with more inter-compartmentalization communications and global variable accesses. The average overhead from compartmentalization is 2.09% and 2.60% in ArduPilot and Jackal, aligning with the findings of prior work that we adopted in our implementation. The task with the highest overhead is ekf_check (8.7%) in ArduPilot and AddRangeData (5.50%) in Jackal, which can be attributed to its responsibility for fusing intermediate results from different sensors, thus introducing frequent compartment switches between ekf_check, ComputeConstaints and sensor input compartments.

Attack Detection Overhead. The runtime overhead from attack detection is from memory corruption attack detection mechanisms, i.e., value-based DFI [54] and CFI [79]. The critical variables protected by value-based DFI in our prototype include conditional variables and variables used in digital controllers. As a result, the average runtime overhead introduced by attack detection mechanisms increases with more indirect jumps and critical variable accesses in each task. The average runtime overhead of tasks in ArduPilot and Jackal is 4.66% and 4.19%, respectively. The tasks with the highest attack detection overhead are landinggear_update (5.69%) and AddRangeData (4.99%), which can be attributed to their control computation complexity, requiring a high frequency of critical control variable accesses.

Checkpointing Latency. The one-time latency of checkpointing snapshots arises from saving the memory state after CPS initialization, increasing proportionally with the size of the memory states to be saved. Specifically, taking a snapshot



(a) Runtime overhead (b) Memory overhead

Figure 8: Runtime and memory overhead of Jackal UGV. Figure 8a shares the same legend with Figure 7.

takes 79 ms on ArduPilot and 16 ms on Jackal, which is negligible compared to the CPS initialization times of 3.13 seconds on ArduPilot and 6.19 seconds on Jackal.

Memory Overhead. Figure 8b shows the memory overhead of software components in Gecko on ArduPilot and Jackal. Overall, the total memory overhead for ArduPilot is 10.85 MB, while the available RAM is 1 GB. Regarding Jackal, the memory overhead amounts to 184.42 MB, with the total RAM size being 16 GB. The total memory overhead consists of four parts. (1) Gecko maintains the uncorrected memory state after system initialization, which introduces the most significant part of the memory overhead. Specifically, Gecko introduces 43.27% and 91.61% memory overhead on ArduPilot and Jackal, respectively. (2) Gecko alters the memory layout and instruments the CPS for the enforcement of compartmentalization policies, introducing 31.9% memory overhead on ArduPilot and 17.2% on Jackal. (3) Gecko adds instrumentation to enforce memory corruption detection, resulting in 16.38% and 13.04% memory overhead on ArduPilot and Jackal, respectively.

6.2 Effectiveness of Gecko under Attacks

Attack Emulation. From the CWE (software weaknesses ranking) [88], the most prevalent memory vulnerabilities are out-of-bounds write, use-after-free, and OS command injection. We selected out-of-bounds write as the attack vector. The other two attacks are similar for testing recovery, as the only difference is that they require different detection strategies. For the ArduPilot platform, we inject a vulnerability into its user interface components based on the real-world CVE-2022-28711. For the Jackal, due to the lack of a real-world CVE, we inject a vulnerability into the function handling user input, which manages waypoint assignment. The attack is then launched at a random time after the mission starts. Once triggered, it launches an out-of-bounds write on the actuation signals, altering the throttle value sent to the actuators. To emulate repeated attacks, the attack is reactivated at random intervals between 0 and 1 second after the initial activation, where the interval is sufficient to disrupt system stability. To emulate a sophisticated attack, we assume the attacker can craft a payload that bypasses detection in the input task but is detected in the next downstreaming task, which is ekf_check in ArduPilot and AddImuData in Jackal, respectively.

Metric. To evaluate the effectiveness of Gecko, we consider two aspects: (1) software availability, quantified by the number of deadline hits during recovery, and (2) control performance, quantified by the deviation between the actual trajectory and the reference trajectory over time.

Task Deadline Hits under Attacks. We measured the recovery time from when an attack is detected to when the program resumes execution after restoring its state. Over 10 runs, the average recovery time was 15.1 ms for ArduPilot and 35.4 ms for Jackal. With such response time, only 4 tasks out of 39 tasks (89.7% deadline hit rate) in ArduPilot and 1 task out of 6 tasks (83.3% deadline hit rate) in Jackal miss deadlines due to their high execution frequencies. Moreover, the task that missed the deadline only happens for one instance, with no consecutive deadline misses.

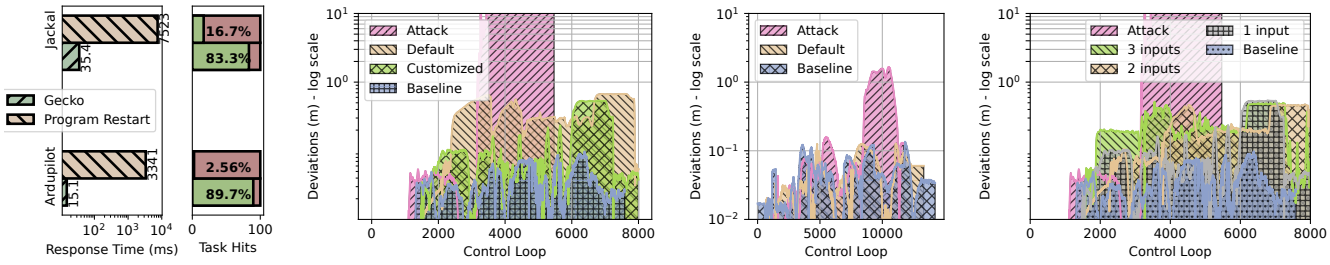
Compared to existing recovery methods such as system reboot (which results in a 0% deadline hit rate for both applications) and program restart (which achieves only 2.56% and 16.7% deadline hit rates for ArduPilot and Jackal, respectively), Gecko significantly reduces missed deadlines and enhances the availability of real-time software. The results are plotted in Figure 9a.

Shadow Compartment Effectiveness. The results with and without shadow compartments are plotted in Figures 9d, 9b, and 9c. Under normal conditions, the control deviations have an average of 0.12 m (meters) on ArduPilot and 0.03 m on Jackal. When under attack, the average deviation is 39.98 m in ArduPilot and 0.51 m in Jackal.

Default policy. We use the default policy to handle attacks originating from user inputs in both ArduPilot and Jackal. On both platforms, user input handling is well-encapsulated as individual components. In ArduPilot, this is managed by the Mavlink module [89], while in Jackal, it is handled by a single thread implemented using ROS [90]. By employing shadow compartments that return default values, the control performance remains unaffected. These default shadow compartments are sufficient to ensure the system stays operational and successfully completes its mission.

Customized Policy. To further assess the effectiveness of applying customized policies in the shadow compartment, we also consider scenarios where sensor inputs are compromised.

We assume that the IMU data is compromised on both platforms. Under these circumstances, we observe that the default policy fails on ArduPilot but remains functional on Jackal. This difference arises because Jackal primarily relies on LiDAR point clouds for control, whereas ArduPilot depends heavily on IMU data for UAV attitude estimation. Thus, Jackal could complete the mission with a deviation of 0.04 m.



(a) Deadline hits. (b) Shadow cmpt. on ArduPilot. (c) Shadow cmpt. on Jackal. (d) Different inputs disabled.

Figure 9: Evaluation of Gecko's Effectiveness.

In contrast, ArduPilot maintains the mission for only a short duration, experiencing a deviation of 0.17 m and returning to the landing point.

To address this, we implemented a proof-of-concept shadow compartment for ArduPilot to compensate for missing data. Specifically, the customized policy employs a record-and-replay approach: sensor data is recorded in advance under normal conditions. Upon an attack, the shadow compartment replays this data to approximate the missing sensor input.

Figure 9b shows the control deviations over time. The results indicate that the customized shadow compartment achieves better performance than the default version, reducing trajectory deviation to 0.12 m in ArduPilot. Additionally, the customized shadow compartment enables the UAV to complete the entire mission rather than landing prematurely. However, this improved effectiveness comes at the cost of increased manual effort in development.

Scalability. The performance degradation is closely related to the disabled sensor inputs. Therefore, we evaluate the effectiveness of the shadow compartment in compensating for various missing inputs. Specifically, we consider three possible inputs that could be disabled in ArduPilot: the GPS signal, gyroscope, and accelerometer.

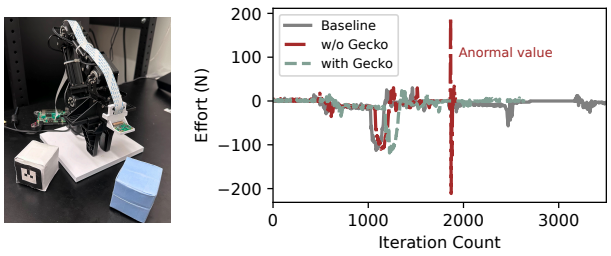
Figure 9d illustrates the reduction in trajectory deviation during a mission when different numbers of inputs are disabled. In the case where one input is disabled, the accelerometer is disabled. When two inputs are disabled, both the accelerometer and gyroscope are disabled. With three inputs disabled, the accelerometer, gyroscope, and GPS are all disabled. The results show that the average trajectory deviations in ArduPilot are 0.118 m with one input disabled, 0.168 m with two inputs disabled, and 0.198 m with three inputs disabled. These results also indicate that reducing the number of disabled inputs through attack surface identification can enhance overall control performance.

6.3 Real-world Case Study

This case study demonstrates the effectiveness of Gecko on a real-world CPS, the OpenManipulator X robotic arm [27], with the physical device shown in Figure 10a. The robotic arm is programmed to execute a predefined mission emulating an industrial workflow: it detects the position of a box by recognizing its QR code, grasps the box, and places it at the designated destination. Additionally, the robotic arm provides an interface for users to modify the mission, such as interrupting tasks or changing the destination of objects. To emulate an attack, a buffer overflow vulnerability is intentionally inserted into the function that processes user input. This vulnerability is triggered during the mission, allowing the attacker to overwrite the wait time between two waypoints [10]. This results in the robotic arm's joints exerting excessive force, causing it to collide forcefully with the table. After the initial attack, the attacker continuously carries out repeated attacks.

Results. Figure 10b illustrates the force exerted by the robotic arm's end-effector over multiple control iterations. Under normal conditions, the force changes gradually and remains consistently below 120 N (Newtons). However, during an attack, the force can increase sharply within a short period, reaching up to 200 N. This sudden surge not only disrupts the original mission but also causes the robotic arm to crash into the table. We deploy Gecko on the robotic arm and configure the shadow compartment with a function that resets the arm to its initial pose. We repeated the experiment. During this run, Gecko detected the illegal write operations and prevented them. It then disabled the compartment responsible for processing user inputs. Since this compartment does not handle essential data or code for control, its deactivation only affects operability. Subsequently, the software rolls back to its initial state, which takes 8.71 ms. Since the recovered program state does not include the mission data, the shadow compartment responds by resetting the robotic arm to its initial pose. This recovery process effectively prevents damage caused by the attack while ensuring the system's safety and stability.

To evaluate the impact of Gecko under normal conditions, we measured its performance overhead. The results show that Gecko introduces a runtime overhead of 7.97% and a memory overhead of 61.4 MB on the OpenManipulator X system configured with four compartments. Despite these overheads, the system successfully ensured that all real-time



(a) Setup. (b) Joint efforts with/without attack.

Figure 10: Case study on OpenManipulator X robotic arm.

tasks met their deadlines.

7 Security Analysis

In this work, we decompose real-time software availability into liveness and timeliness in the context of system recovery. As such, the analysis would focus on these two aspects as well as the realization of the protection mechanism.

Liveness Protection. An attacker may attempt to repeatedly exploit the same vulnerability to compromise the availability. However, Gecko prevents this by identifying and isolating potentially vulnerable input compartments immediately after an exploit is detected, thereby thwarting further exploitation. Since the attack vector attribution has no false negatives (although false positives may occur), the vulnerable input vector is guaranteed to be removed. While the reduction of attack surface from disabling the potentially vulnerable compartments degrades performance, it is mitigated by the shadow compartments, which compensate for the missing features and maintain system functionality. However, it is important to note that even though eventual execution is guaranteed, depending on the number of stages configured in the multi-staged input compartment attribution, there is an unavoidable trade-off on how quickly the vulnerability is mitigated and how accurate the defense action is. The more stages there are, the finer-grained the compartments are without introducing prohibitive overhead, however, this also leads to more rounds of attribution when the attacker can continue to cause program security violations, impacting the system availability.

Timeliness Protection. The timeliness aspect, on the other hand, focuses on how quickly Gecko can clean up the attack impact (caveat on the effectiveness of the security monitor), and the time to clean up has to be bounded. The current design leverages selective checkpoint and restore function, which scales according to the size of the program memory, and therefore should be bounded. However, the recovery to the initialized value may not always be the secure/safe choice. For example, setting altitude to 0, as initialized, while the drone is in flight may cause the system to destabilize. In our current implementation, this has to be addressed by the shadow compartment. Further discussion is available in Section 8.

Meta-data Corruption Prevention. Gecko utilizes runtime-saved metadata to enforce compartmentalization, detect memory attacks, and facilitate system state recovery. However, attackers may seek to undermine Gecko's effectiveness by corrupting this metadata to bypass detection or disrupt the recovery process. Gecko prevents this attack by storing all metadata in a dedicated memory region, isolated from the application compartments using software fault isolation techniques. By sandboxing the compartments, Gecko ensures that unauthorized memory access is prevented, maintaining the integrity of the metadata.

8 Limitations and Future Work

Gecko is an ambitious attempt to tackle the long-standing challenge of availability assurance. It has many limitations that require further study, including cyber-physical state inconsistency in recovery and shadow compartments in feature reduction.

Cyber-physical Inconsistency in Recovery. The current design of Gecko leverages the checkpoint/restore technique to achieve timeliness in recovery. By using the initialized state as a clean slate, the recovery process eliminates the impact of the attack. However, many control software are inherently stateful, and directly recovering the cyber state to initialization will also remove the valuable control states, leading to a significant discrepancy between the cyber representation of the physical states and the actual values and poor control performance. To minimize the impact of recovery, more checkpoints can be set up throughout the program. Upon recovery, selected states (spatial) can be rolled back to different checkpoints (temporal). Further investigation is needed on how to ensure the absence of adversarial data in these checkpoints, as well as the consistency among the recovered and non-recovered states. Besides the fidelity of the checkpoints and recovery mechanism, inconsistency (both among the cyber states and cyber-physical states) will always exist, necessitating techniques to quickly recognize and mitigate the inconsistencies. Our current implementation builds on top of existing approaches in control software for handling intermittent faults. For example, ArduPilot [25] re-initializes IMU trackers upon detecting inconsistencies, and the Jackal [91] re-initializes its odometry system when the location is lost. In cases where no such re-initialization method exists in the original codebase, it can be a non-trivial effort that would require significant domain expertise to develop the mitigation.

Shadow Compartment. Gecko utilizes shadow compartments as a safe mode to compensate for disabled functionality. However, its current support for shadow compartments remains limited. Specifically, the default mode is designed for non-essential inputs, such as debugging interfaces, which

means that many scenarios require the development of a customized mode. While this customized shadow compartment can often be built on top of fail-safe routines in software, its design presents fundamental trade-offs between control performance, security, and simplicity. Logically complex algorithms can often achieve better control performance. However, sophisticated controllers often take more manual effort to develop and fine-tune. Furthermore, the complexity can often lead to a higher probability of bugs and vulnerability with the larger TCB. For this very reason, the Boeing 777 control system [92] incorporates two subsystems to ensure redundancy: one for high performance and the other for reliability. The primary subsystem offers enhanced functionality across various conditions to maximize performance, while the secondary subsystem is developed via certifiable Ada compilers, activating only in the event of a failure in the primary subsystem. This Simplex design is also the basis for the shadow compartments in Gecko. One direction to strike a better balance in this trade-off is to leverage formal methods to improve the reliability and security of the alternative controller while keeping the development effort affordable. Furthermore, the development of shadow compartments presents a unique set of challenges compared to fail-safe handlers, where input can potentially be adversarially manipulated. Gecko took a simplistic approach of disabling the input vector, an additional study is needed to understand the feasibility of opportunistically enabling the input again while bounding the availability and control performance of the system.

Generality of Gecko. Even though Gecko is designed for CPS, many of the elements can be generalized to other application domains where real-time system availability is important. A recent survey study [64] reveals that many compartmentalized applications, including web and datacenter servers [93, 94], require a high degree of availability. Particularly, we believe there are opportunities in unifying the timeliness notion in Gecko with the QoS objective in server applications. Furthermore, servers are often highly elastic, presenting new opportunities to track, survive, and recover from DoS attacks.

Interface Safety in Compartmentalization. The adoption of compartmentalization in Gecko's design introduces potential interface safety issues [47], where the attacker may launch confused deputy attacks [46]. Currently, Gecko mitigates these risks by leveraging physical-world semantics to sanitize the data exchanged between compartments. However, additional efforts are needed to study the feasibility of automatic generation of the privileges of each interface.

Compatibility with Hardware-based Isolation Mechanisms. The compartment isolation in Gecko adopts the same model as prior works [64, 95], where the subject is a unit of computation (e.g., an assembly instruction, a thread of execution) and the object is a unit of privilege enforcement (e.g., a byte of memory, a socket). This model allows Gecko to be retrofitted to be compatible with hardware-based isolation mechanism as Intel MPK [96] and ARM memory domains [97] as these hardware features allow permissions to be defined for objects based on specific subjects. However, these hardware features are limited in the number of compartments they can support (e.g., a maximum of 16 in Intel MPK [96] and fewer in ARM memory domains [97]) compared to software-based compartmentalization.

9 Conclusion

In this work, we formulated the property of real-time software availability, encompassing both liveness and timeliness properties, and propose Gecko to achieve it. To ensure liveness, Gecko automatically identifies the attack vector and disables the exploited features in the system. Gecko leverages compartmentalization to enable a trade-off between attack attribution granularity and performance. To achieve timeliness, Gecko leverages the predictability in sensing and actuation of CPS software to selectively reset program states in CPSs, avoiding peripheral reconfiguration. We built the prototype and evaluated it on three CPS applications, demonstrating the feasibility and also highlighting the limitations that require future efforts from the community.

Acknowledgment

We thank the reviewers for their valuable feedback. This work was partially supported by NSF (CNS-2038995, CNS-2154930, CNS-2229427, CNS-2238635, CNS-2403758), ARO (W911NF-24-1-0155), and Intel.

10 Ethics Considerations and Compliance with Open Science Policy

Ethics Consideration. This work primarily focuses on defensive technologies for cyber-physical systems. We do not anticipate that these defensive technologies could be leveraged to introduce attacks. Furthermore, no new vulnerabilities are discovered in this paper; the vulnerabilities used in our evaluation are publicly known CVEs.

Open Science. To facilitate reproducibility, we openly share our research artifacts[4]. Our system, Gecko, is composed of three main components: (1) compartmentalization and instrumentation, (2) shadow compartment mechanism, and (3) checkpoint/restore mechanisms. The artifact provides all the compiler passes needed for compartmentalization and instrumentation and the code for checkpoint/restore. Since the shadow compartment is target-specific and requires manual efforts to develop, we include an example implementation in ArduPilot.

References

[1] D. Kaplan, J. Powell, and T. Woller, "Amd memory encryption," *White paper*, vol. 13, 2016.

[2] M. Abadi, M. Budiu, Ú. Erlingsson, and J. Ligatti, "Control-flow integrity," in *Proceedings of the 12th ACM conference on Computer and communications security*, 2005, pp. 340–353.

[3] M. Castro, M. Costa, and T. Harris, "Securing software by enforcing data-flow integrity," in *Proceedings of the 7th symposium on Operating systems design and implementation*, 2006, pp. 147–160.

[4] Y. Wang, A. Li, J. Wang, S. Baruah, and N. Zhang, "Opportunistic data flow integrity for real-time cyber-physical systems using worst case execution time reservation," in *Proceedings of the 33rd USENIX Conference on Security Symposium*. USENIX Association, 2024.

[5] G. Candea, S. Kawamoto, Y. Fujiki, G. Friedman, and A. Fox, "Microreboot — a technique for cheap recovery," in *6th USENIX Symposium on Operating Systems Design and Implementation (OSDI 2004)*, 2004.

[6] V. Narayanan, T. Huang, D. Detweiler, D. Appel, Z. Li, G. Zellweger, and A. Burtsev, "{RedLeaf}: Isolation and communication in a safe operating system," in *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, 2020, pp. 21–39.

[7] A. Depoutovitch and M. Stumm, "Otherworld: giving applications a chance to survive os kernel crashes," in *Proceedings of the 5th European conference on Computer systems*, 2010, pp. 181–194.

[8] M. M. Swift, M. Annamalai, B. N. Bershad, and H. M. Levy, "Recovering device drivers," in *Proceedings of the 6th Conference on Symposium on Operating Systems Design & Implementation - Volume 6*, ser. OSDI'04. USA: USENIX Association, 2004, p. 1.

[9] A. Li and N. Zhang, "Data-flow availability: Achieving timing assurance in autonomous systems," in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, 2024, pp. 445–463.

[10] A. Li, J. Wang, S. Baruah, B. Sinopoli, and N. Zhang, "An empirical study of performance interference: Timing violation patterns and impacts," in *Proceedings of the 30th IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS'24)*. IEEE Computer Society Press, 2024.

[11] F. Qin, J. Tucek, J. Sundaresan, and Y. Zhou, "Rx: treating bugs as allergies—a safe method to survive software failures," in *Proceedings of the twentieth ACM symposium on Operating systems principles*, 2005, pp. 235–248.

[12] W. Zhang, M. De Kruijf, A. Li, S. Lu, and K. Sankaralingam, "Conair: Featherweight concurrency bug recovery via single-threaded idempotent execution," in *Proceedings of the eighteenth international conference on Architectural support for programming languages and operating systems*, 2013, pp. 113–126.

[13] R. Koo and S. Toueg, "Checkpointing and rollback-recovery for distributed systems," *IEEE Transactions on software Engineering*, no. 1, pp. 23–31, 1987.

[14] F. Kong, M. Xu, J. Weimer, O. Sokolsky, and I. Lee, "Cyber-physical system checkpointing and recovery," in *2018 ACM/IEEE 9th International Conference on Cyber-Physical Systems (ICCPS)*. IEEE, 2018, pp. 22–31.

[15] R. Smith and S. Rixner, "Surviving peripheral failures in embedded systems," in *2015 USENIX Annual Technical Conference (USENIX ATC 15)*, 2015, pp. 125–137.

[16] J. Tucek, S. Lu, C. Huang, S. Xanthos, and Y. Zhou, "Triage: diagnosing production run failures at the user's site," in *Proceedings of twenty-first ACM SIGOPS symposium on Operating systems principles*, 2007, pp. 131–144.

[17] J. Newsome and D. X. Song, "Dynamic taint analysis for automatic detection, analysis, and signaturegeneration of exploits on commodity software." in *12th Annual Network And Distributed System Security Symposium (NDSS 2005)*, 2005.

[4] The artifact is available at https://zenodo.org/records/14751667.

[18] T. Kim, X. Wang, N. Zeldovich, and M. F. Kaashoek, "Intrusion recovery using selective re-execution," in *9th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2010.

[19] S. T. King and P. M. Chen, "Backtracking intrusions," in *Proceedings of the Nineteenth ACM Symposium on Operating Systems Principles*, 2003, p. 223–236.

[20] S. T. King, Z. M. Mao, D. G. Lucchetti, and P. M. Chen, "Enriching intrusion alerts through multi-host causality." in *12th Annual Network And Distributed System Security Symposium (NDSS 2005)*, 2005.

[21] S. Ma, X. Zhang, and D. Xu, "Protracer: Towards practical provenance tracing by alternating between logging and tainting," in *23rd Annual Network And Distributed System Security Symposium (NDSS 2016)*, 2016.

[22] T. Kim, C. H. Kim, A. Ozen, F. Fei, Z. Tu, X. Zhang, X. Deng, D. J. Tian, and D. Xu, "From control model to program: Investigating robotic aerial vehicle accidents with {MAYDAY}," in *29th USENIX Security Symposium (USENIX Security 20)*, 2020, pp. 913–930.

[23] Y. Ji, S. Lee, M. Fazzini, J. Allen, E. Downing, T. Kim, A. Orso, and W. Lee, "Enabling refinable {Cross-Host} attack investigation with efficient data flow tagging and tracking," in *27th USENIX Security Symposium (USENIX Security 18)*, 2018, pp. 1705–1722.

[24] K. H. Lee, X. Zhang, and D. Xu, "High accuracy attack provenance via binary-based execution partition," in *20th Annual Network And Distributed System Security Symposium (NDSS 2013)*, vol. 16, 2013.

[25] "Ardupilot project," https://ardupilot.org/, accessed: 2022-10-02.

[26] Clearpath Robotics, "Jackal unmanned ground vehicle," https://clearpathrobotics.com/jackal-small-unmanned-ground-vehicle/, accessed: 2023-12-25.

[27] ROBOTIS, "Openmanipulator x," https://emanual.robotis.com/docs/en/platform/openmanipulator_x/overview/, accessed: 2023-9-12.

[28] B. Akesson, M. Nasri, G. Nelissen, S. Altmeyer, and R. I. Davis, "An empirical survey-based study into industry practice in real-time systems," in *2020 IEEE Real-Time Systems Symposium (RTSS)*. IEEE, 2020, pp. 3–11.

[29] PX4 Development Team, "Px4 autopilot," https://px4.io/, [Accessed 2024-01-29].

[30] M. Costa, J. Crowcroft, M. Castro, A. Rowstron, L. Zhou, L. Zhang, and P. Barham, "Vigilante: End-to-end containment of internet worms," in *Proceedings of the twentieth ACM symposium on Operating systems principles*, 2005, pp. 133–147.

[31] M. T. I. Ziad, M. A. Arroyo, E. Manzhosov, and S. Sethumadhavan, "Zerø: Zero-overhead resilient operation under pointer integrity attacks," in *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2021, pp. 999–1012.

[32] A. Rösti, S. Volckaert, M. Franz, and A. Voulimeneas, "I'll be there for you! perpetual availability in the a8 mvx system," in *Proceedings of the ACSAC 2024 - Annual Computer Security Applications Conference*, San Antonio, TX, USA, 2024.

[33] F. Fei, Z. Tu, R. Yu, T. Kim, X. Zhang, D. Xu, and X. Deng, "Cross-layer retrofitting of uavs against cyber-physical attacks," in *2018 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2018, pp. 550–557.

[34] M. Arroyo, H. Kobayashi, S. Sethumadhavan, and J. Yang, "Fired: frequent inertial resets with diversification for emerging commodity cyber-physical systems," *arXiv preprint arXiv:1702.06595*, 2017.

[35] Y. Huang, C. Kintala, N. Kolettis, and N. D. Fulton, "Software rejuvenation: Analysis, module and applications," in *Twenty-fifth international symposium on fault-tolerant computing. Digest of papers*. IEEE, 1995, pp. 381–390.

[36] L. Sha, "Using simplicity to control complexity," *IEEE Software*, vol. 18, no. 4, pp. 20–28, 2001.

[37] S. Bak, D. K. Chivukula, O. Adekunle, M. Sun, M. Caccamo, and L. Sha, "The system-level simplex architecture for improved real-time embedded system safety," in *2009 15th IEEE Real-Time and Embedded Technology and Applications Symposium*. IEEE, 2009, pp. 99–107.

[38] M. Costa, M. Castro, L. Zhou, L. Zhang, and M. Peinado, "Bouncer: Securing software by blocking bad input," in *Proceedings of twenty-first ACM SIGOPS symposium on Operating systems principles*, 2007, pp. 117–130.

[39] J. Newsome, D. Brumley, D. Song, J. Chamcham, and X. Kovah, "Vulnerability-specific execution filtering for exploit prevention on commodity software." in *20th Annual Network And Distributed System Security Symposium (NDSS 2013)*, 2006.

[40] F. Abdi, C.-Y. Chen, M. Hasan, S. Liu, S. Mohan, and M. Caccamo, "Guaranteed physical security with restart-based design for cyber-physical systems," in *2018 ACM/IEEE 9th International Conference on Cyber-Physical Systems (ICCPS)*. IEEE, 2018, pp. 10–21.

[41] F. M. David, E. Chan, J. C. Carlyle, and R. H. Campbell, "Curios: Improving reliability through operating system structure." in *OSDI*, vol. 8, 2008, pp. 59–72.

[42] T. Gu, C. Sun, X. Ma, J. Lü, and Z. Su, "Automatic runtime recovery via error handler synthesis," in *Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering*, 2016, pp. 684–695.

[43] S. Boyd-Wickizer and N. Zeldovich, "Tolerating malicious device drivers in linux," in *2010 USENIX Annual Technical Conference (USENIX ATC 10)*, 2010.

[44] C. Yan, H. Shin, C. Bolton, W. Xu, Y. Kim, and K. Fu, "Sok: A minimalist approach to formalizing analog sensor security," in *2020 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2020, pp. 233–248.

[45] P. Kocher, "Differential power analysis," in *Proc. Advances in Cryptology (CRYPTO'99)*, 1999.

[46] H. Lefeuvre, V.-A. Badoiu, Y. Chien, F. Huici, N. Dautenhahn, and P. Olivier, "Assessing the impact of interface vulnerabilities in compartmentalized software," in *30th Annual Network And Distributed System Security Symposium (NDSS 2023)*, 2023.

[47] H. Hu, Z. L. Chua, Z. Liang, and P. Saxena, "Identifying arbitrary memory access vulnerabilities in privilege-separated software," in *Computer Security–ESORICS 2015: 20th European Symposium on Research in Computer Security, Vienna, Austria, September 21-25, 2015, Proceedings, Part II 20*. Springer, 2015, pp. 312–331.

[48] A. Li, M. Sudvarg, H. Liu, Z. Yu, C. Gill, and N. Zhang, "Polyrhythm: Adaptive tuning of a multi-channel attack template for timing interference," in *2022 IEEE Real-Time Systems Symposium (RTSS)*. IEEE, 2022, pp. 225–239.

[49] C. H. Kim, T. Kim, H. Choi, Z. Gu, B. Lee, X. Zhang, and D. Xu, "Securing real-time microcontroller systems through customized memory view switching." in *25th Annual Network And Distributed System Security Symposium (NDSS 2018)*, 2018.

[50] C. Song, B. Lee, K. Lu, W. Harris, T. Kim, and W. Lee, "Enforcing kernel security invariants with data flow integrity." in *23rd Annual Network And Distributed System Security Symposium (NDSS 2016)*, 2016.

[51] L. Szekeres, M. Payer, T. Wei, and D. Song, "Sok: Eternal war in memory," in *2013 IEEE Symposium on Security and Privacy*. IEEE, 2013, pp. 48–62.

[52] D. Song, J. Lettner, P. Rajasekaran, Y. Na, S. Volckaert, P. Larsen, and M. Franz, "Sok: Sanitizing for security," in *2019 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2019, pp. 1275–1295.

[53] N. Burow, S. A. Carr, J. Nash, P. Larsen, M. Franz, S. Brunthaler, and M. Payer, "Control-flow integrity: Precision, security, and performance," *ACM Computing Surveys (CSUR)*, vol. 50, no. 1, pp. 1–33, 2017.

[54] Z. Sun, B. Feng, L. Lu, and S. Jha, "Oat: Attesting operation integrity of embedded devices," in *2020 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2020, pp. 1433–1449.

[55] E. Std, "Railway applications-communications, signalling and processing systems-safety related electronic systems for signalling," *European Committee for Electrotechnical Standardisation (CENELEC)*, 2019.

[56] S. P. Assurance, "Techniques for radiation effects mitigation in aasic and fpgas handbook," Technical report, ESA Requirements and Standards Division, Tech. Rep., 2016.

[57] A. Avižienis, J.-C. Laprie, and B. Randell, "Dependability and its threats: a taxonomy," in *Building the Information Society: IFIP 18th World Computer Congress Topical Sessions 22–27 August 2004 Toulouse, France*. Springer, 2004, pp. 91–120.

[58] ArduPilot Dev Team, "Ekf2 estimation system," https://ardupilot.org/dev/docs/ekf2-estimation-system.html, 2024, [Online; accessed 28-December-2024].

[59] ArduPilot, "Failsafe landing page," n.d., accessed: 2024-12-14. [Online]. Available: https://ardupilot.org/copter/docs/failsafe-landing-page.html

[60] PX4 Community, "Safety configuration documentation," https://docs.px4.io/main/en/config/safety.html, 2024, accessed: 2024-12-26.

[61] ROS Community, "Move base parameters documentation," https://wiki.ros.org/move_base#Parameters, 2024, accessed: 2024-12-26.

[62] M. Contributors, "Moveit2 plan execution source code," https://github.com/moveit/moveit2/blob/e2d9dc56134b167e5faae1d061b96c38c1ad2e64/moveit_ros/planning/plan_execution/src/plan_execution.cpp#L161, 2024, accessed: 2024-12-26.

[63] A. A. Clements, N. S. Almakhdhub, S. Bagchi, and M. Payer, "{ACES}: Automatic compartments for embedded systems," in *27th USENIX Security Symposium (USENIX Security 18)*, 2018, pp. 65–82.

[64] H. Lefeuvre, N. Dautenhahn, D. Chisnall, and P. Olivier, "Sok: Software compartmentalization," in *2025 IEEE Symposium on Security and Privacy (SP)*, 2024.

[65] J. Ferrante, K. J. Ottenstein, and J. D. Warren, "The program dependence graph and its use in optimization," *ACM Transactions on Programming Languages and Systems (TOPLAS)*, vol. 9, no. 3, pp. 319–349, 1987.

[66] G. Parmer and R. West, "Mutable protection domains: Towards a component-based system for dependable and predictable computing," in *28th IEEE International Real-Time Systems Symposium (RTSS 2007)*. IEEE, 2007, pp. 365–378.

[67] C. Maiza, H. Rihani, J. M. Rivas, J. Goossens, S. Altmeyer, and R. I. Davis, "A survey of timing verification techniques for multi-core real-time systems," *ACM Computing Surveys (CSUR)*, vol. 52, no. 3, pp. 1–38, 2019.

[68] D. Quarta, M. Pogliani, M. Polino, F. Maggi, A. M. Zanchettin, and S. Zanero, "An experimental security analysis of an industrial robot controller," in *2017 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2017, pp. 268–286.

[69] Z. Huang, M. DAngelo, D. Miyani, and D. Lie, "Talos: Neutralizing vulnerabilities with security workarounds for rapid response," in *2016 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2016, pp. 618–635.

[70] J. Jeong, D. Kim, J.-H. Jang, J. Noh, C. Song, and Y. Kim, "Un-rocking drones: Foundations of acoustic injection attacks and recovery thereof." in *30th Annual Network And Distributed System Security Symposium (NDSS 2023)*, 2023.

[71] H. Choi, S. Kate, Y. Aafer, X. Zhang, and D. Xu, "Software-based realtime recovery from sensor attacks on robotic vehicles," in *23rd International Symposium on Research in Attacks, Intrusions and Defenses (RAID 2020)*, 2020, pp. 349–364.

[72] R. E. Kalman, "A new approach to linear filtering and prediction problems," 1960.

[73] P. Dash, G. Li, Z. Chen, M. Karimibiuki, and K. Pattabiraman, "Pid-piper: Recovering robotic vehicles from physical attacks," in *2021 51st Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. IEEE, 2021, pp. 26–38.

[74] P. Dash, E. Chan, and K. Pattabiraman, "Specguard: Specification aware recovery for robotic autonomous vehicles from physical attacks," in *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, 2024, pp. 1849–1863.

[75] S. S. Mukherjee, M. Kontz, and S. K. Reinhardt, "Detailed design and evaluation of redundant multithreading alternatives," *ACM SIGARCH Computer Architecture News*, vol. 30, no. 2, pp. 99–110, 2002.

[76] A. Kadav, M. J. Renzelmann, and M. M. Swift, "Fine-grained fault tolerance using device checkpoints," *ACM SIGPLAN Notices*, vol. 48, no. 4, pp. 473–484, 2013.

[77] V. Izosimov, P. Pop, P. Eles, and Z. Peng, "Design optimization of time-and cost-constrained fault-tolerant distributed embedded systems," in *Design, Automation and Test in Europe*. IEEE, 2005, pp. 864–869.

[78] Y. Sui and J. Xue, "Svf: interprocedural static value-flow analysis in llvm," in *Proceedings of the 25th international conference on compiler construction*, 2016, pp. 265–266.

[79] LLVM Team, "Control flow integrity - clang documentation," accessed: 2024-01-22. [Online]. Available: https://clang.llvm.org/docs/ControlFlowIntegrity.html

[80] C. in Userspace (CRIU) Project, "Criu: Checkpoint/restore in userspace," accessed: 2023-11-23. [Online]. Available: https://github.com/checkpoint-restore/criu

[81] Emlid, "Navio2 documentation," https://docs.emlid.com/navio2/, accessed: 2023-12-15.

[82] SICK, "LMS1xx LiDAR Sensor," https://www.sick.com/au/en/catalog/products/lidar-and-radar-sensors/lidar-sensors/lms1xx/c/g91901, 2023, accessed on 2023-12-25.

[83] "Moveit," https://moveit.ros.org/, accessed: 2024-01-02.

[84] "Ros-industrial applications of moveit," https://moveit.ros.org/robots/, accessed: 2024-01-03.

[85] "Picknik robotics wins space force, nasa contracts," https://www.therobotreport.com/picknik-robotics-wins-space-force-nasa-contracts/, accessed: 2022-10-23.

[86] X. Shi, L. Cao, D. Wang, L. Liu, G. You, S. Liu, and C. Wang, "Hero: Accelerating autonomous robotic tasks with fpga," in *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2018, pp. 7766–7772.

[87] C. L. Liu and J. W. Layland, "Scheduling algorithms for multiprogramming in a hard-real-time environment," *Journal of the ACM (JACM)*, vol. 20, no. 1, pp. 46–61, 1973.

[88] MITRE Corporation, "CWE Top 25 Most Dangerous Software Weaknesses - 2023," 2023, accessed: 2024-08-16. [Online]. Available: https://cwe.mitre.org/top25/archive/2023/2023_top25_list.html

[89] "MAVLink – Micro Air Vehicle Message Marshalling Library," https://mavlink.io/, accessed: 2023-12-25.

[90] "Ros (robot operating system)," https://www.ros.org/, 2024, accessed: 2024-08-28.

[91] Google, "Google cartographer documentation," https://google-cartographer.readthedocs.io/en/latest/, 2023, accessed on 2023-12-15.

[92] Y. C. Yeh, "Triple-triple redundant 777 primary flight computer," in *1996 IEEE Aerospace Applications Conference. Proceedings*, vol. 1. IEEE, 1996, pp. 293–307.

[93] The Apache Software Foundation, "Apache http server - worker mpm," https://httpd.apache.org/docs/2.4/mod/worker.html, accessed: 2024-12-13.

[94] Lighttpd Documentation, "Lighttpd2 - core configuration angel," https://doc.lighttpd.net/lighttpd2/core_config_angel.html, accessed: 2024-12-13.

[95] J. Wang, Y. Wang, A. Li, Y. Xiao, R. Zhang, W. Lou, Y. T. Hou, and N. Zhang, "{ARI}: Attestation of real-time mission execution integrity," in *32nd USENIX Security Symposium (USENIX Security 23)*, 2023, pp. 2761–2778.

[96] "Intel memory protection keys," https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html, 2024, accessed: 2024-12-16.

[97] "Mpu memory region," https://developer.arm.com/documentation/107565/0101/Memory-protection/Memory-Protection-Unit, 2024, accessed: 2024-12-16.